Quantizing Neural Networks for Real-Time Physics Triggers

An easy, detailed study guide to the CERN / HEP project

FP32 vs INT8 / INT6 / INT4 | PTQ vs QAT | hls4ml & FPGA | Pareto analysis

How to use this guide
Read it in order - each chapter builds on the previous one. Every chapter ends with a short **Key takeaway** and the guide ends with a glossary and self-test questions. Numbers in examples are **illustrative** unless stated otherwise.

Contents

1. The project in one sentence
2. The CERN problem: too many collisions
3. The trigger and why ML helps
4. Why neural networks are expensive
5. Quantization: the core idea
6. Quantization can hurt accuracy
7. The trade-off map
8. PTQ vs QAT
9. HEP data and jet tagging
10. FPGAs and hls4ml
11. The environment: why WSL2
12. The full architecture
13. What you are (and are not) claiming
14. The experiment matrix and Pareto frontier
15. Test-twice development philosophy
16. Phase 1 deliverables and the 10-phase roadmap
17. Telling the CERN story
18. Glossary
19. Self-test questions

1. The project in one sentence

This project is a **research benchmarking platform**. It studies how much we can compress (quantize) the neural networks used for high-energy-physics (HEP) event classification, while keeping their physics performance, and making them cheap and fast enough for real-time trigger systems.

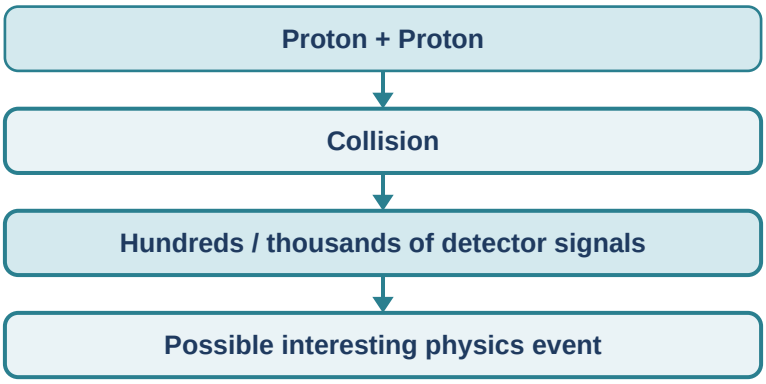
In plain words
Can we make an AI model used in particle physics **much smaller and faster** without making it **significantly worse**?

That single question is the heart of everything. The rest of this guide explains each word in it: *model, smaller, faster, worse, particle physics*.

Key takeaway
You are not just training a model. You are measuring a **trade-off** between accuracy and cost.

2. The CERN problem: too many collisions

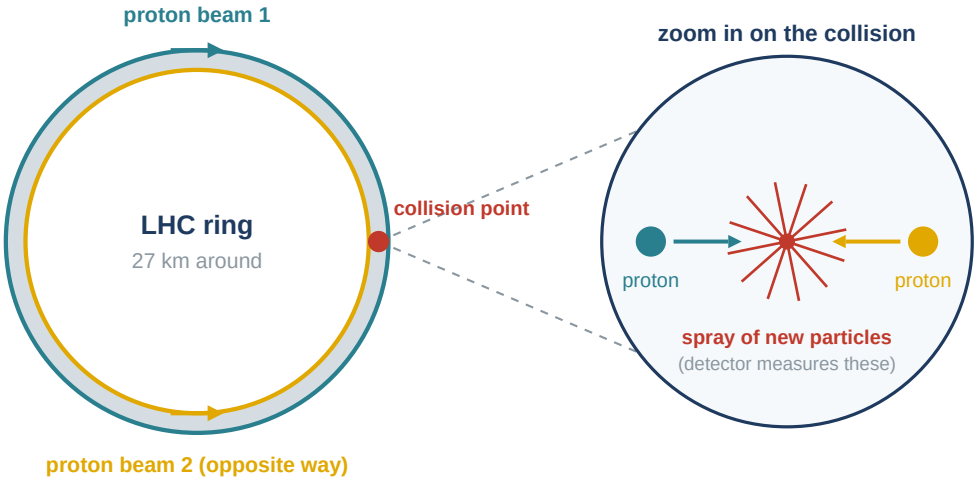
At the Large Hadron Collider (LHC), protons are smashed together at enormous rates. Each collision produces hundreds or thousands of detector measurements.



The problem: there is **far too much data** to store every collision in full detail. Most collisions are boring (known physics). A tiny fraction might hold something new or important. We must decide, extremely quickly, which to keep.

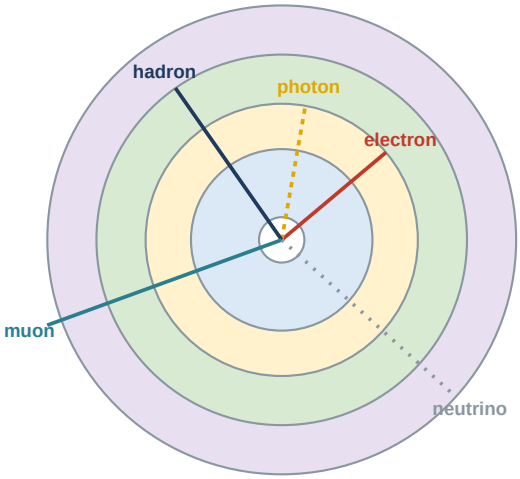
Where collisions happen: the LHC

The LHC is a 27 km underground ring. Two beams of protons race around it in opposite directions at almost the speed of light and are steered to cross at a few points. A **collision** is when two protons hit; the energy turns into a spray of new particles.



How we see it: the detector

Around each collision point sits a giant detector built in layers, like an onion. Different particles stop in different layers, so the pattern of signals tells us what came out.



cross-section of a detector (like an onion)

Tracker
Records curved paths of charged particles; the bend tells us their momentum.

ECAL (electromagnetic calorimeter)
Stops electrons & photons and measures their energy.

HCAL (hadronic calorimeter)
Stops hadrons (pions, protons...) and measures their energy. Jets mostly land here.

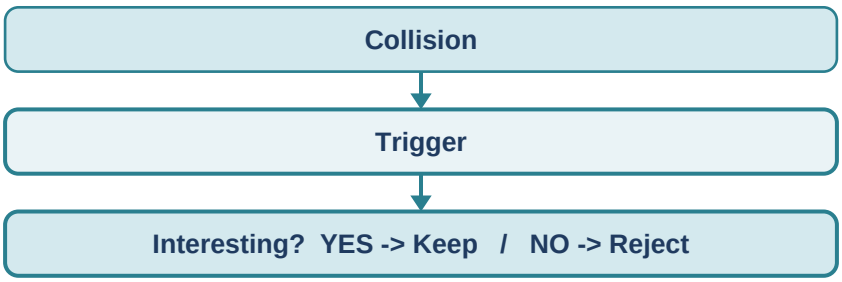
Muon chambers
Only muons get this far - they pass through everything.

Neutrino
Invisible: leaves no signal at all, seen only as 'missing' energy.

Everyday analogy
Imagine a camera taking millions of photos per second but with room to save only a few. A smart filter must instantly decide which photos are worth keeping.

3. The trigger and why ML helps

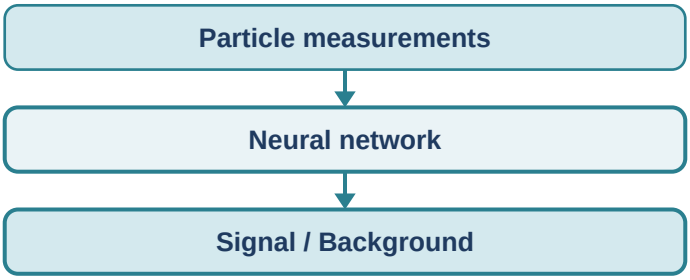
The **trigger system** is that filter. It asks one question for every collision: *"Is this event interesting enough to keep?"*



Traditionally, triggers use hand-written rules. **Machine learning** can instead *learn patterns from previous events*. A neural network can combine many measurements at once:

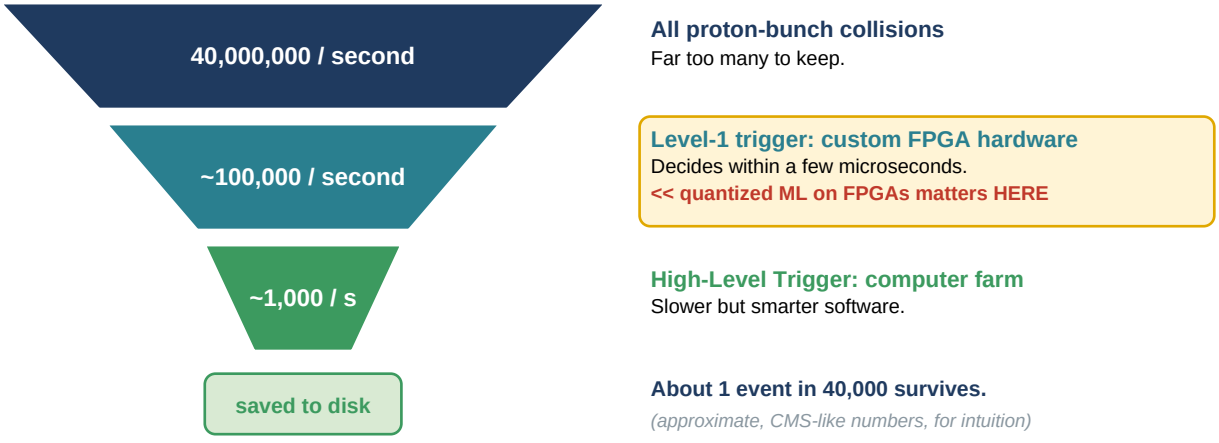
- particle momentum and energy
- particle type
- spatial information (where things hit the detector)
- jet constituents (the particles inside a jet)
- other detector measurements

The network outputs a classification such as **signal** (interesting) vs **background** (boring). This can give better event selection than fixed rules.



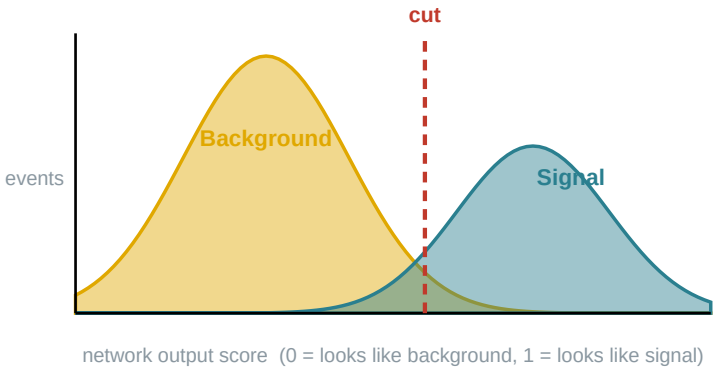
The trigger as a funnel

The trigger works in stages, each one throwing away most of what is left. Only the first stage is fast enough to need custom hardware (FPGAs).



Signal vs background

Signal = the rare events we want. **Background** = the common boring events that look similar. The network gives each event a score and we keep it if the score passes a cut.



How a classifier decides

Each event gets a score from the network. Keep the event if score > cut.

The two hills overlap, so mistakes happen:
- background above the cut = false positive
- signal below the cut = false negative

A better model = less overlap.

Key takeaway
ML can pick interesting events better than hand-made rules - **but only if it is fast enough**. That is the catch of the next chapter.

4. Why neural networks are expensive

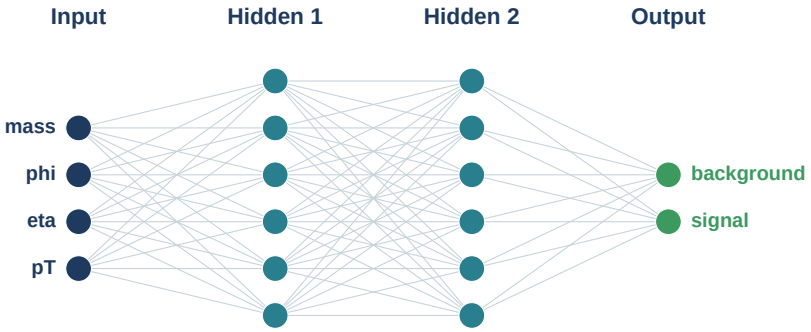
A normal neural network may have around **1,000,000 parameters** (numbers it learned) and perform millions of arithmetic operations per prediction. On a GPU, server or cloud machine that is fine. A real-time trigger is different:

- It cannot wait - decisions are needed in a tiny fraction of a second (nanoseconds to microseconds).
- It does not have unlimited hardware.
- It must give predictable, guaranteed timing.

So we face a fundamental engineering trade-off:

If the model gets more complex...	...then
Better representation	Potentially better accuracy
More computation	Slower (higher latency)
More memory	Bigger, more expensive hardware
More hardware resources	Might not fit on the chip at all

What is inside the network



Each line is one WEIGHT
(a learned number).

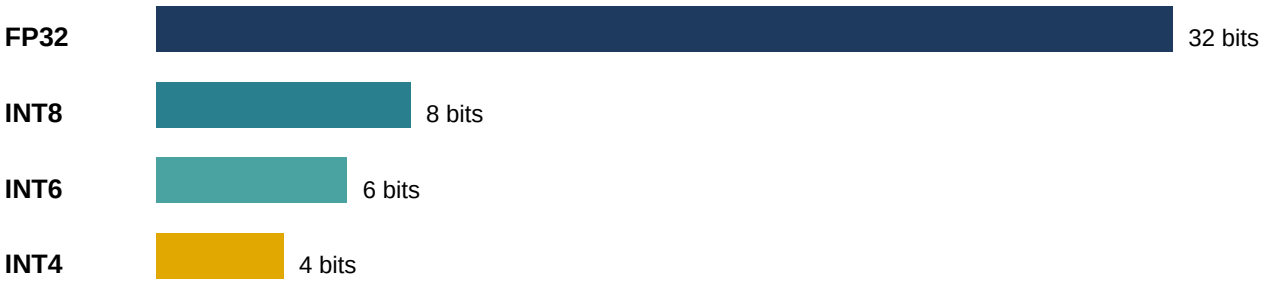
FP32 stores each weight
in 32 bits; INT4 in 4 bits.

1,000,000 weights:
FP32 -> ~4 MB
INT4 -> ~0.5 MB

Key takeaway
Accuracy and cost pull in opposite directions. **Your project attacks this tension.**

5. Quantization: the core idea

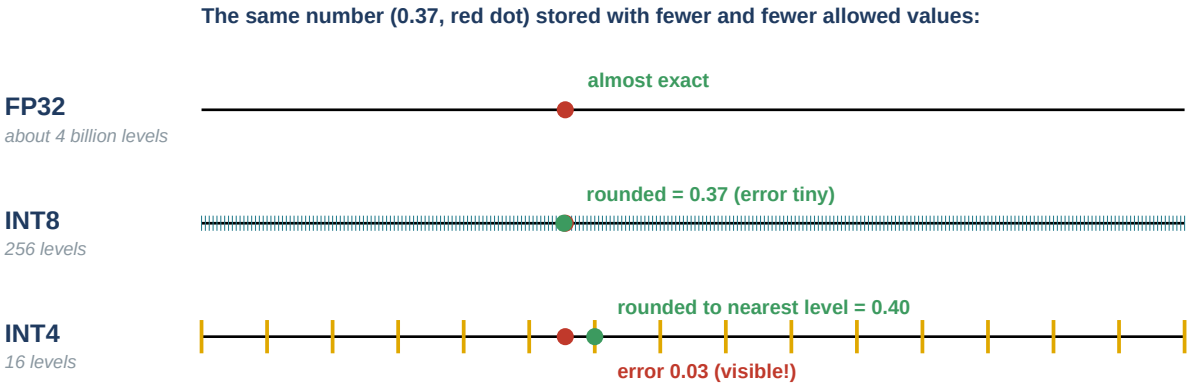
Quantization means storing and computing numbers with fewer bits. A normal network uses **FP32** (32-bit floating-point numbers). We try smaller formats:



Format	Bits	Meaning
FP32	32	Full-precision floating point. Very precise, expensive. The baseline.
INT8	8	8-bit integers: 256 possible values per number.
INT6	6	6-bit integers: 64 possible values.
INT4	4	4-bit integers: only 16 possible values.

Fewer bits can make the model **smaller**, **cheaper to compute**, **more hardware-friendly** and potentially **faster**.

Rounding on a number line



Analogy
Describing a colour with a 16-million-colour palette vs a 16-colour palette. The small palette is far cheaper to store, but you lose subtle shades. If the picture is simple, you may not notice. That is exactly the question: *does the physics notice?*

6. Quantization can hurt accuracy

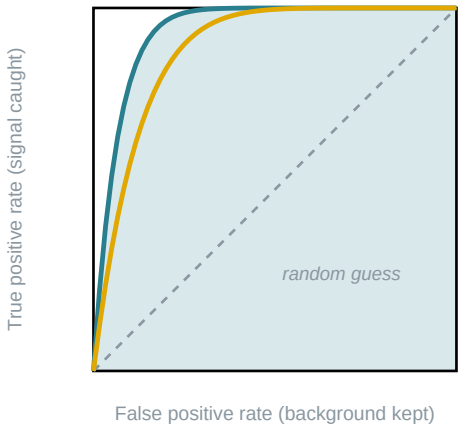
The catch: rounding numbers into fewer values loses information. Performance is measured here mainly by **ROC-AUC** (1.0 = perfect separation, 0.5 = coin flip). Illustrative example:

Precision	ROC-AUC	Verdict
FP32	0.940	Baseline

Precision	ROC-AUC	Verdict
INT8	0.939	Tiny loss - probably fine
INT4	0.890	Large loss - maybe unacceptable

So the real research question is not "is INT8 faster?" (that is obvious). It is:

ROC curve and AUC explained



Reading a ROC curve

Slide the cut from strict to loose and plot how much signal you catch vs how much background sneaks in.

AUC = area under the curve (shaded).

FP32 model, AUC ~ 0.94 (curve hugs top-left)
badly quantized model, AUC ~ 0.89
random guess, AUC = 0.5

1.0 = perfect 0.5 = coin flip

The research question
How much compression can we get before physics performance becomes unacceptable?

7. The trade-off map

Your experiments will fill a table like this. **These numbers are made-up examples**, only to show the idea:

Model	Precision	ROC-AUC	Size	Latency*	FPGA resources
MLP	FP32	0.940	2.1 MB	100 ns	High
MLP	INT8	0.938	0.55 MB	55 ns	Medium
MLP	INT6	0.934	0.42 MB	48 ns	Lower
MLP	INT4	0.901	0.28 MB	35 ns	Low

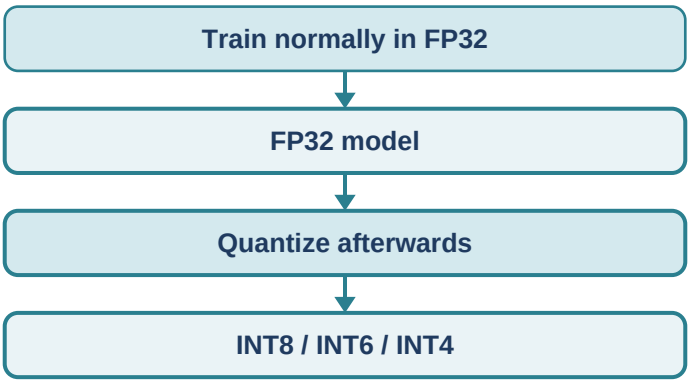
** Illustrative only - not project results.*

Now you can ask the central analytic question: **is the small accuracy loss worth the reduction in cost?** Here INT8 and INT6 cost almost no accuracy but save a lot; INT4 saves more but drops noticeably.

8. PTQ vs QAT

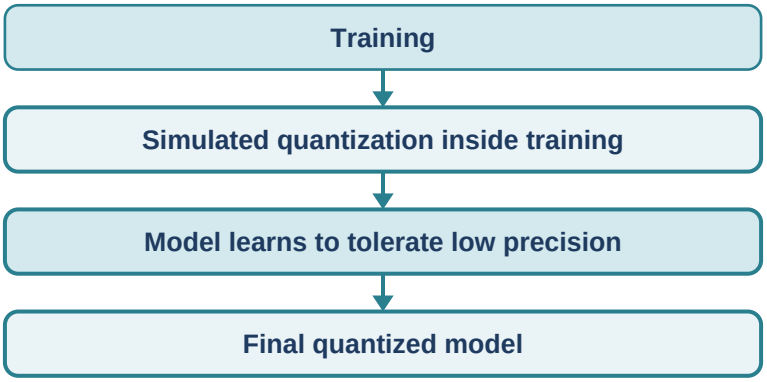
You do not only compare bit widths. You also compare **two ways of getting to low precision**.

Strategy 1 - PTQ (Post-Training Quantization)



Simple and fast. But the model never learned to cope with rounding, so accuracy can drop, especially at 4 bits.

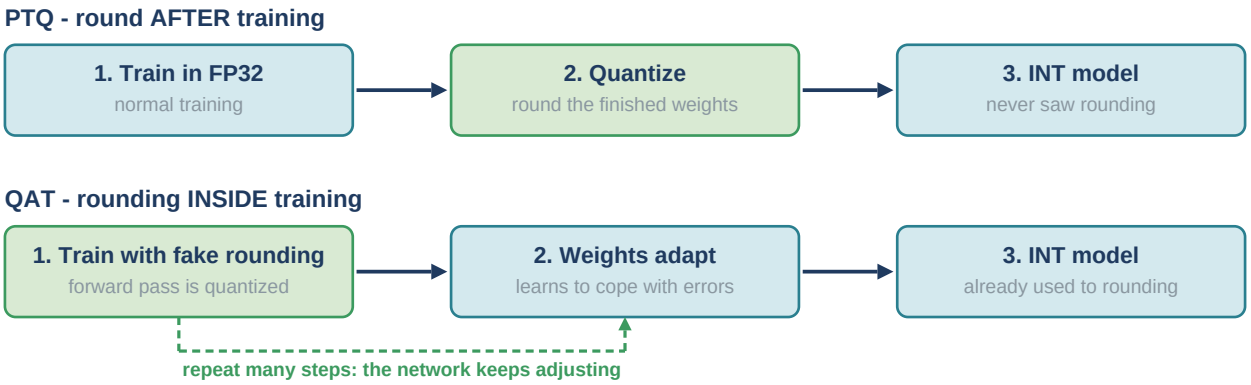
Strategy 2 - QAT (Quantization-Aware Training)



More work, but the network adapts to the rounding errors while learning, so it usually keeps more accuracy.

	PTQ	QAT
When quantized	After training	During training
Effort	Low	Higher (retrain)
Accuracy at low bits	Often drops	Usually recovers more
Analogy	Shrinking a finished photo	Painting directly in a small palette

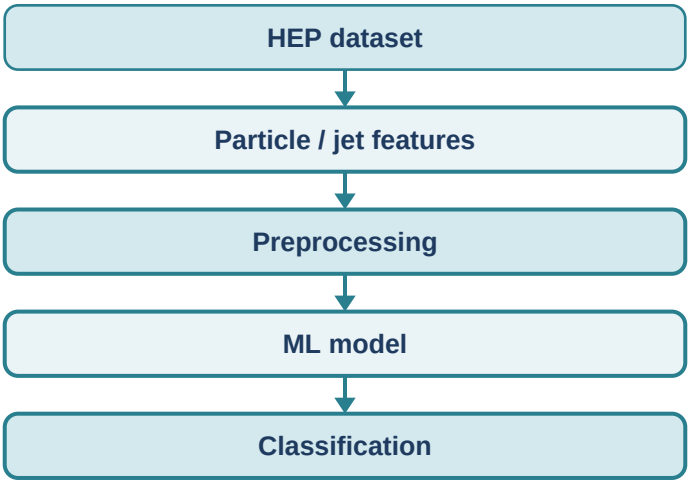
PTQ and QAT side by side



The experimental question
Can QAT recover some of the accuracy that PTQ loses? You test FP32 baseline plus {PTQ, QAT} x {INT8, INT6, INT4} = 6 quantized models.

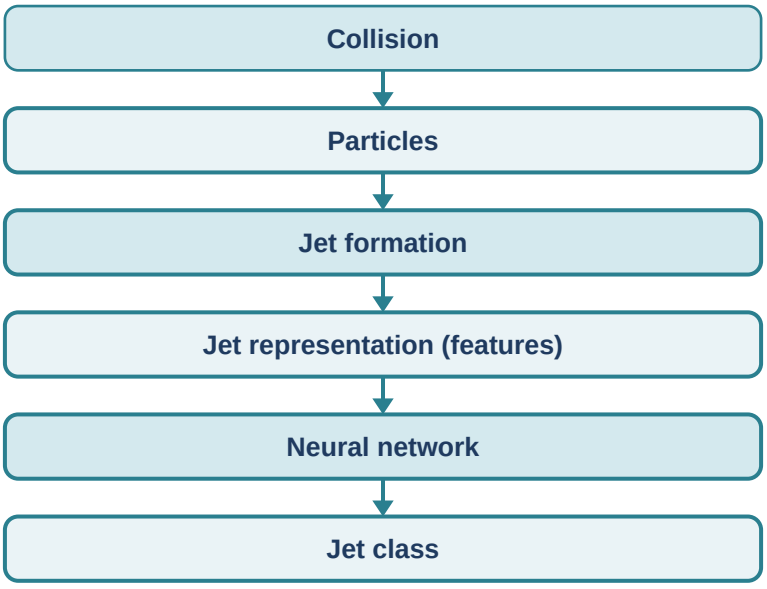
9. HEP data and jet tagging

This is not a generic compression exercise on MNIST or CIFAR. You use a **real High-Energy-Physics dataset**, so the result is meaningful for science.



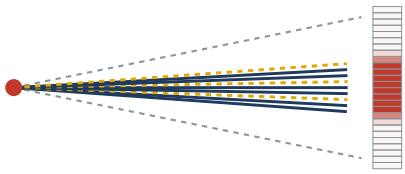
What is a jet?

A **jet** is a narrow spray of particles produced when a high-energy quark or gluon (or other particle) flies out of a collision. Different origin particles leave different jet patterns - this is why a network can tell them apart. **Jet tagging** = classifying which kind of particle produced a jet.

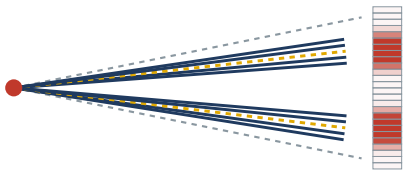


Jets as pictures

Quark / gluon jet: ONE dense core



W / top jet: 2+ separate cores



<- collision calorimeter strip: redder = more energy ->

What is a jet?

A quark or gluon cannot travel alone, so it turns into a narrow spray of particles heading the same way. That spray is a JET.

Solid lines: charged particles (tracker sees)
Dashed lines: neutral particles

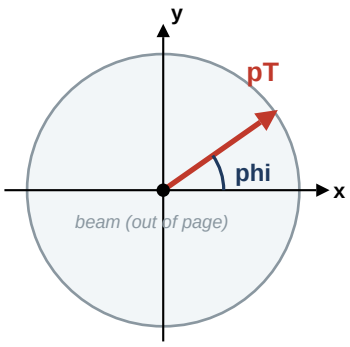
Jet tagging = deciding what started the jet.

Different parents -> different shapes:
- width of the spray
- number of particles
- how many separate cores
- total mass of the spray

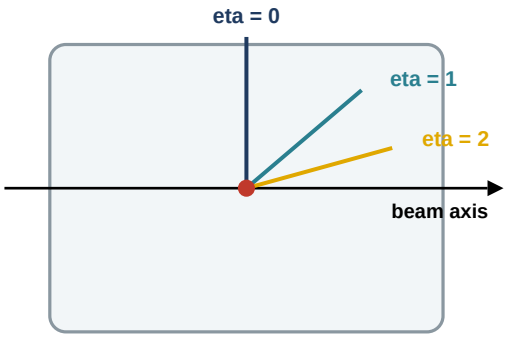
The numbers the network sees (pT, eta, phi)

A jet is described to the network by a few measured quantities. The most common are **pT** (transverse momentum), **eta** and **phi** (where the particle went), plus energy and mass.

End view (looking down the beam)



Side view



pT = momentum sideways to the beam phi = direction angle around the beam
eta = angle from the beam axis: eta = 0 sideways (central), large eta = close to the beam (forward)

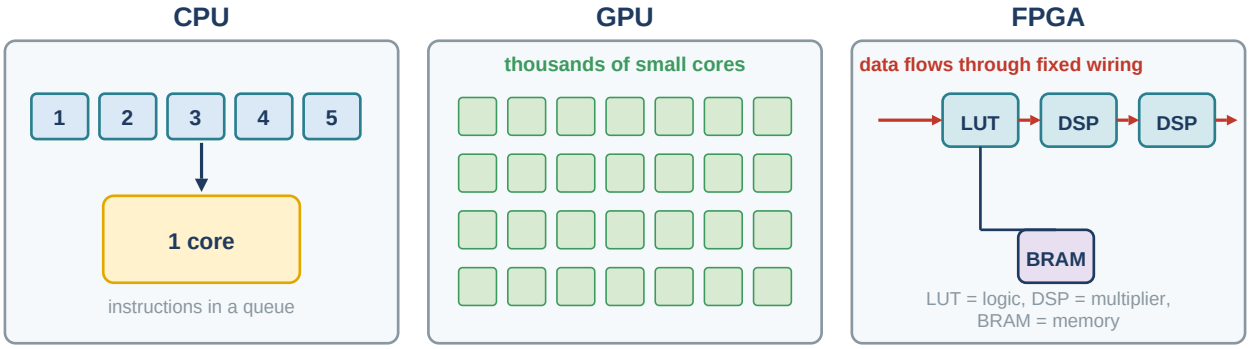
Key takeaway
Jet tagging is a well-known, realistic HEP ML task - an ideal testbed for your quantization study.

10. FPGAs and hls4ml

What is an FPGA?

A CPU executes instructions one after another. A GPU runs many operations in parallel. An **FPGA** (Field-Programmable Gate Array) is a chip whose **hardware logic itself is configured** to match your computation. This gives **very predictable, ultra-low timing** - which is why triggers use them.

Device	Mental model	Strength
CPU	Follows instructions	Flexible
GPU (e.g. your RTX 5060)	Thousands of parallel workers	High throughput
FPGA	Custom-built circuit	Tiny, predictable latency



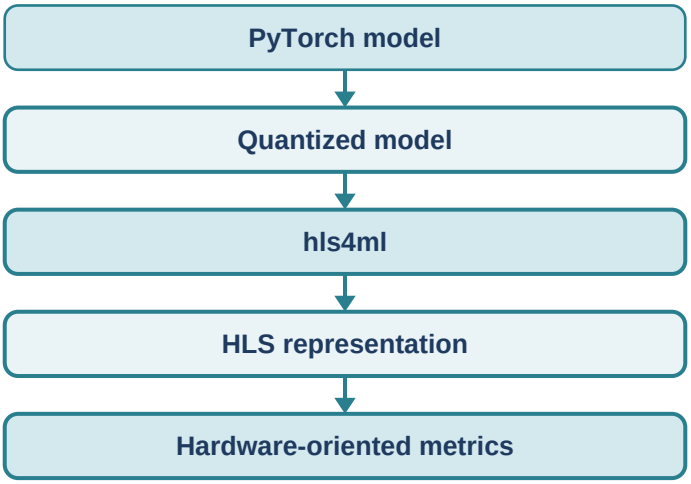
flexible, one step at a time | *huge throughput, timing varies* | *circuit shaped like your network*

Why quantization matters for FPGAs

A 32-bit floating-point multiplier needs far more chip area than an 8-bit or 4-bit integer one. Fewer bits -> fewer chip resources -> faster and smaller designs.

What is hls4ml?

hls4ml is a toolchain that translates certain ML models into High-Level-Synthesis (HLS) code for FPGA workflows.

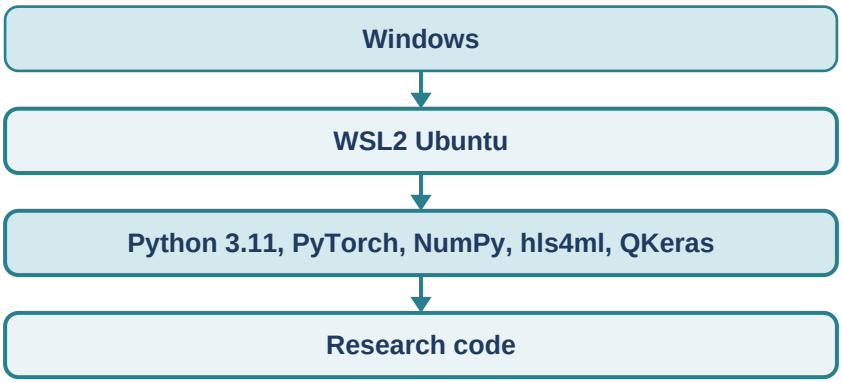


Metric	Meaning
Latency	Time from input to output
Initiation interval (II)	How often a new input can be accepted
LUT	Look-up tables - the general logic of the chip
DSP	Dedicated multiplier blocks
BRAM	On-chip memory blocks
FF	Flip-flops - small storage elements

Honesty rule
Your laptop does **not** give physical FPGA measurements. Without a real FPGA or full synthesis flow you must label results as **software estimates / simulation / synthesis output**, never as measured hardware numbers. (Your repo already has a three-tier hardware backend including a SoftwareEstimateBackend for exactly this reason.)

11. The environment: why WSL2

Windows Smart App Control was blocking Matplotlib's compiled extension. Instead of weakening Windows security, the project moves into **WSL2** (Windows Subsystem for Linux), which gives a Linux environment ideal for scientific and HLS tooling.



12. The full architecture

Read this as a pipeline from top to bottom. Each stage feeds the next.

Stage	What happens	Why it matters
1. Public HEP data	Download a public dataset	Real science data
2. Dataset validation	Check NaN, infinity, labels, leakage	Bad data = meaningless results
3. Preprocessing	Scale / clean features	Stable training
4. FP32 baseline	Train full-precision network	The reference to compare against
5. Baseline evaluation	AUC, accuracy, F1	Know the starting point
6. PTQ and QAT	Make INT8 / INT6 / INT4 models both ways	The experiment itself
7. Evaluation	Accuracy, F1, AUC, size, latency	Measure everything
8. hls4ml analysis	Hardware-oriented estimates	Link to FPGA reality
9. Pareto analysis	Accuracy vs cost / latency / resources	Find the best trade-offs
10. Research report	Write up honestly	The deliverable

What is leakage?
Data **leakage** means information from the test set sneaks into training, making results look better than they are. Validating for it is part of rigorous science.

13. What you are (and are not) claiming

This matters a lot for credibility.

Do NOT claim	DO claim
"I solved CERN's trigger problem."	"I investigated an accuracy / cost / hardware trade-off relevant to ML-based real-time HEP triggering."
"I trained a net and got 94% accuracy."	"I systematically evaluated the accuracy-precision-latency-resource trade-off of quantized networks for HEP event classification."
Physical FPGA numbers from a laptop	Clearly labelled software estimates / synthesis outputs

Your contribution is the **benchmarking methodology and empirical analysis**. You are studying a *system*, not merely training a model.

Questions your study answers
How much can we compress? -> How much accuracy is lost? -> How much compute is saved? -> What happens to hardware resources? -> Which configurations lie on the Pareto frontier?

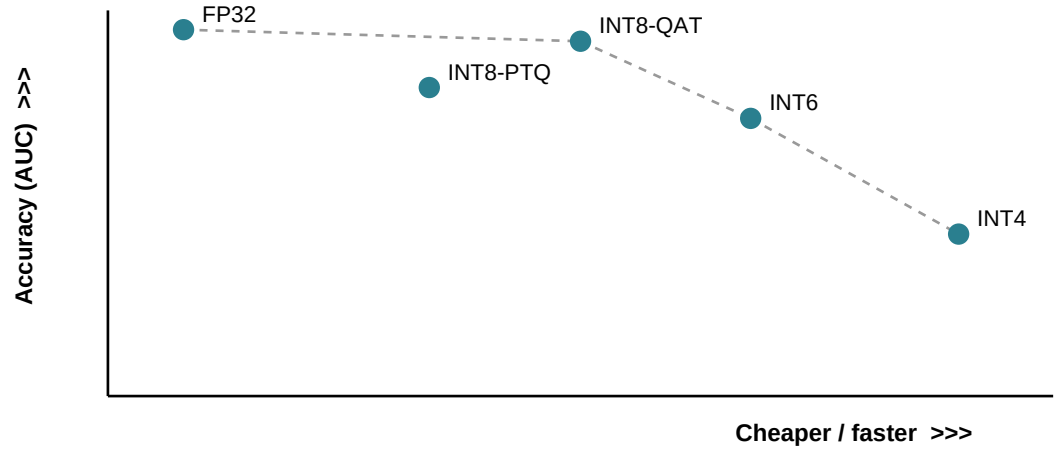
14. The experiment matrix and Pareto frontier

Architecture	Method	Precision	AUC	Latency	Resources
MLP	FP32	FP32	...	...	...
MLP	PTQ	INT8	...	...	...
MLP	PTQ	INT6	...	...	...
MLP	PTQ	INT4	...	...	...
MLP	QAT	INT8	...	...	...
MLP	QAT	INT6	...	...	...
MLP	QAT	INT4	...	...	...

Later you can add other architectures (CNN, ParticleNet-style) - but not yet. Start with the MLP and do it properly.

The Pareto frontier

A configuration is on the **Pareto frontier** if no other configuration is both more accurate *and* cheaper. The frontier is the set of "best possible trade-offs".

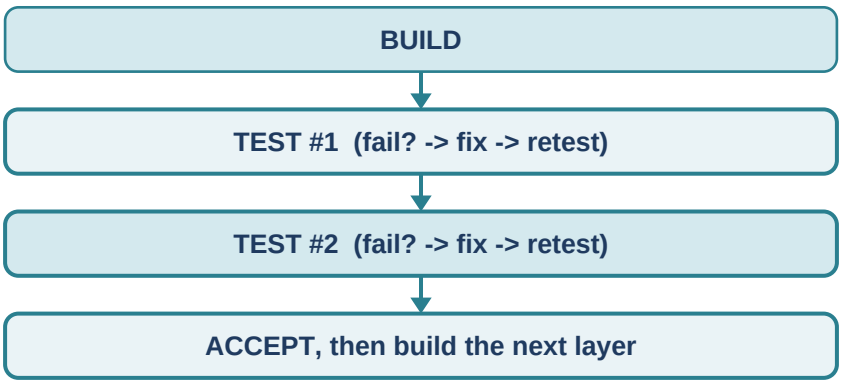


Illustrative shape only - not project results. Dashed line = Pareto frontier.

The wrong question is "which model is best?". The right question is: **which configurations offer different useful trade-offs between physics performance and cost?** The answer depends on how tight the trigger's latency budget is.

15. Test-twice development philosophy

To avoid "demo-ware" (code that looks good but is unreliable), every step is verified **before** building on it.



Example order: dataset loader -> tests pass twice -> FP32 training -> tests pass twice -> PTQ -> tests pass twice -> QAT. If you stack features on an unstable base, you can never tell where a bug came from.

Key takeaway
Small verified steps beat one giant unverified build.

16. Phase 1 deliverables and the 10-phase roadmap

What a good first build contains

Area	Deliverables
Foundation	Clean Python project, WSL2 env, reproducible dependencies, dataset loader, validation, tests
ML	FP32 baseline, proper metrics, saved checkpoint, reproducible training
Quantization	PTQ framework, supported precision experiments, comparison framework
Research infra	Experiment registry, result CSV, plots, Pareto analysis, report
Hardware	hls4ml abstraction/integration, honest hardware estimates

Quality beats quantity: a few well-tested, reproducible experiments are worth more than twenty sloppy models. QAT can follow once the foundation is solid.

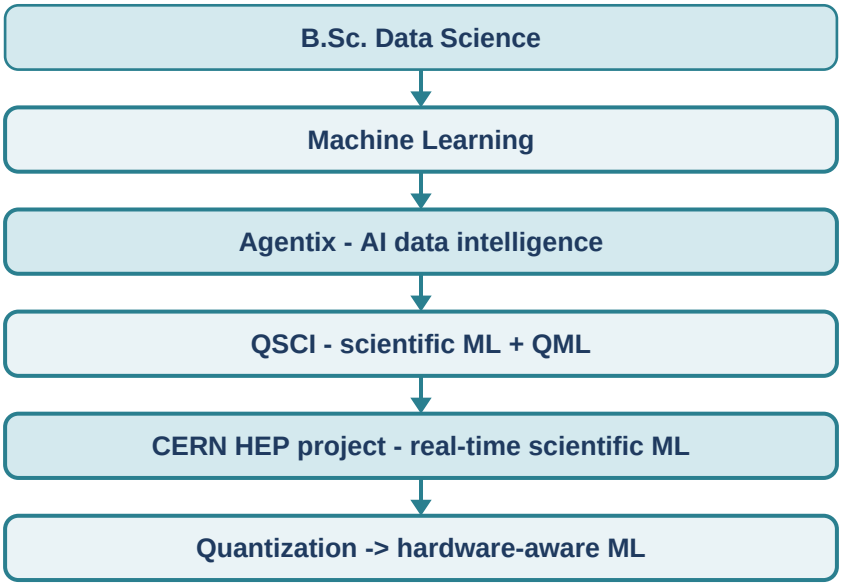
Roadmap

Phase	Goal
1	Engineering foundation
2	Real HEP dataset validation
3	FP32 baseline reproduction
4	PTQ experiments
5	QAT

Phase	Goal
6	hls4ml / FPGA analysis
7	Architecture comparison
8	Second dataset
9	Statistical validation
10	Research paper / technical report

17. Telling the CERN story

Your projects connect into one coherent narrative:



A strong summary to use

I developed a reproducible benchmark for studying the accuracy, computational cost, and hardware-oriented trade-offs of quantized neural networks for HEP event classification. I evaluated FP32, PTQ, and QAT configurations and investigated how reduced numerical precision affects model performance and deployment characteristics.

Your GitHub should **prove** it: code + experiments + tests + datasets + configs + results + plots + reproducibility + a technical report.

18. Glossary

Term	Simple meaning
LHC	Large Hadron Collider - CERN's particle accelerator
HEP	High-Energy Physics
Trigger	Ultra-fast filter deciding which collisions to keep
Jet	Spray of particles from a high-energy quark/gluon
Collision	Two protons hit head-on; energy becomes new particles
Detector	Layered instrument that records the particles
Tracker	Innermost layer: curved tracks of charged particles
Calorimeter (ECAL/HCAL)	Layers that stop particles and measure energy
pT	Transverse momentum: momentum sideways to the beam
eta (pseudorapidity)	Angle from the beam axis: 0 = sideways, large = near beam
phi	Direction angle around the beam
Weight	A learned number in the network (one line in the diagram)
Cut / threshold	Score above which an event is kept
Jet tagging	Classifying what produced a jet
Signal / background	Interesting events / boring events
FP32	32-bit floating-point numbers
INT8/6/4	8/6/4-bit integers
Quantization	Using fewer bits to store/compute numbers
PTQ	Post-Training Quantization - quantize after training
QAT	Quantization-Aware Training - train while simulating low precision
ROC-AUC	Score for separating classes; 1.0 perfect, 0.5 random
F1	Balance of precision and recall
Latency	Time to produce one answer
FPGA	Reconfigurable hardware chip with custom circuits
HLS	High-Level Synthesis: code -> hardware design
hls4ml	Tool turning ML models into HLS/FPGA designs
LUT / DSP / BRAM / FF	FPGA resources: logic / multipliers / memory / registers
Pareto frontier	Set of best trade-offs; nothing strictly better in both axes
Leakage	Test information accidentally used in training
WSL2	Linux environment inside Windows

19. Self-test questions

Try answering aloud before checking the chapter.

- Why can't the LHC save every collision?
- What does the trigger decide?
- Why is a 1M-parameter FP32 network a problem in a trigger?
- What do you lose and gain going from FP32 to INT4?
- Explain PTQ and QAT in one sentence each. Which should do better at INT4, and why?
- What is ROC-AUC and what does 0.5 mean?
- Why is an FPGA suited for triggers?
- What does hls4ml do?
- Why must laptop-derived hardware numbers be labelled as estimates?
- What is a Pareto frontier and why ask 'which configurations?' instead of 'which model is best?'
- Why test each stage twice before building the next?
- State the project's core claim without overclaiming.

One-picture summary
HEP data -> neural network -> FP32 baseline -> {PTQ, QAT} x {INT8, INT6, INT4} -> measure accuracy, latency, resources -> Pareto frontier -> scientific conclusion.

The project is not really about quantization. It is about finding the engineering trade-off between ML performance and real-time computational limits in high-energy physics. Quantization is just the experimental tool.